



RateHopper

SECURITY AUDIT REPORT

———— 14 Sep 2026 ————

Reviewed By:
kenzo, Security Researcher
yashar, Security Researcher

Table of Contents

About Shred Security

Protocol Executive Summary

Disclaimer

Risk Classification

Executive Summary

Findings

Appendix A — Deployment Checklist Compliance

Appendix B — Stateful Invariant Fuzzing Report

Final Notes

About Shred Security

Shred Security provides high-quality security audits for blockchain and DeFi protocols across different chains. Our audits consistently uncover high-impact vulnerabilities missed by others, backed by a proven track record of top competition placements and security partnerships with leading protocols.

We also develop open security resources for the broader ecosystem: our [Protocol Deployment Checklist](#) defines baseline deployment-readiness requirements; our [Incident Response Checklist](#) provides a structured framework for protocol incident response from alert to safe harbor; and [HackViz](#) is an interactive platform to learn from, simulate, and visualize past exploits.

Shred Security's mission is to raise the standard of on-chain security through rigorous audits and practically shared security tooling.

Contact: shredsecurity.io

Protocol Executive Summary

RateHopper is an intelligent DeFi platform on Base that helps users optimize borrowing costs and yield returns through Gnosis Safe modules. Users enable RateHopper modules on their Safe; the modules execute atomic operations on behalf of the Safe without taking custody of assets.

PR #16 expands RateHopper from a debt-switching system into a unified **debt + yield** platform. The primary deliverable is `SafeYieldManager` — a single Safe module for concentrated-liquidity LP lifecycle across **Uniswap V3**, **Aerodrome Slipstream** (with optional gauge staking), and **Uniswap V4** (including native ETH pools). Protocol mechanics live in stateless handlers invoked via `delegatecall`; the manager owns basis bookkeeping, performance/collect fees, TWAP-based swap floors, pool allow-listing, and pause / per-protocol enable switches. Atomic `switchLp` closes a full position and opens its replacement, carrying only the cost basis attributable to value redeployed.

The legacy standalone `RatehopperUniV3Positions` module continues serving pre-existing WETH/USDC V3 positions (coexistence, no state migration). On the debt side, handlers were reorganized under `contracts/debt/` with repayment-residue refunds and related cleanups. Critical configuration remains timelock-gated via `CRITICAL_ROLE`; routine allow-list and TWAP seeding use `DEFAULT_ADMIN_ROLE`.

Disclaimer

A smart contract security review cannot guarantee the complete elimination of vulnerabilities. The process is limited by time, available resources, and human expertise, and is intended to identify as many potential issues as reasonably possible. As such, no assurance can be given that all vulnerabilities will be discovered, or that the reviewed smart contracts are entirely secure. To strengthen security over time, follow-up audits, bug bounty programs, and continuous on-chain monitoring are strongly recommended.

Risk Classification

Findings are classified on two independent axes: **Severity** (potential impact if exploited) and **Difficulty** (skill and conditions required to exploit). These two scores together determine remediation priority — a medium-severity issue that is trivially easy to trigger may warrant more urgency than a high-severity issue that requires privileged access.

Level	Severity — What can happen
CRITICAL	Direct loss or permanent lockup of funds, unauthorized minting, governance outcome manipulation, or protocol insolvency.
HIGH	Serious economic harm: theft or permanent lockup of yield/royalties, or temporary asset freezing.
MEDIUM	Disrupts intended operation: griefing, gas theft, unbounded gas use, block stuffing, or contract DoS.
LOW	Limited impact: failure to deliver expected returns without principal loss, or defects that are hard to trigger in practice.
INFORMATIONAL	Non-exploitable notes on clarity, maintainability, gas hygiene, or design tradeoffs.

Level	Difficulty — How hard to exploit
LOW	Well-known technique; public tools exist or can be scripted with minimal effort.
MEDIUM	Attacker must write a custom exploit or have in-depth system knowledge.
HIGH	Requires privileged access, complex technical details, or other weaknesses to be exploited first.

The likelihood × impact matrix below maps severity ratings against each other:

Likelihood \ Impact	Critical	High	Medium	Low
High	Critical	Critical	High	Medium
Medium	Critical	High	Medium	Low
Low	High	Medium	Low	Low

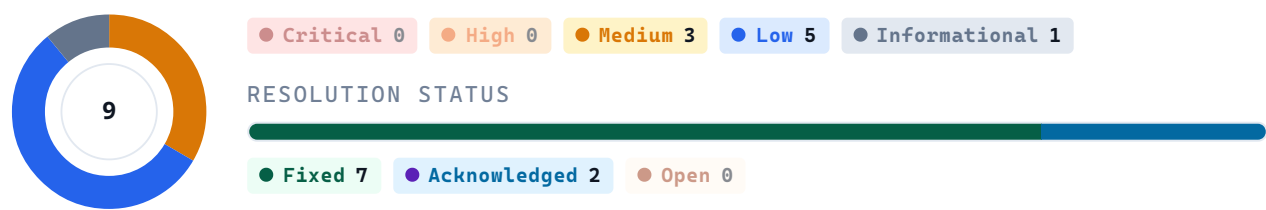
Executive Summary

The Shred Security team reviewed RateHopper PR #16 (`SafeYieldManager` yield stack and related debt-side changes) for 8 days total. A total of **9 issues** were identified: **3 Medium**, **5 Low**, and **1 Informational**. No Critical or High severity issues survived attack-path and impact validation. Remediation was delivered in [PR #19](#) (`fix/audit-shred-20260904`): **7 Fixed**, **2 Acknowledged** (L-2 documented accept; I-1 hardened via timelock), **0 Open**.

Engagement Overview

About the Project		Audit Timeline	
Project Name	Ratehopper	Audit Start	27/08/2026
Repository	github.com/RateHopper/ratehopper-contracts	Audit End	04/09/2026
Contracts	<code>SafeYieldManager.sol</code>	Report Draft	04/09/2026
	, yield handlers,	Report Published	14/09/2026
	<code>MorphoDebtHandler.sol</code>	Remediation	PR #19
	, <code>SafeDebtManager.sol</code>	Effort	8 person-days
Commit	<code>1493cbc</code> / PR #16	Methodology	Manual review + invariant fuzzing
Type	DeFi Safe modules (debt + yield)		
Lines of Code	~2,348 (in-scope Solidity)		

Vulnerability Summary



Severity	Total	Fixed	Acknowledged	Open
CRITICAL	0	—	—	—
HIGH	0	—	—	—
MEDIUM	3	3	0	0
LOW	5	4	1	0
INFORMATIONAL	1	0	1	0
Total	9	7	2	0

Severity × Difficulty Matrix

The matrix below shows where each finding falls. Findings closer to the top-left (Critical/High severity, Low difficulty) demand the most immediate attention.

Severity ↓ / Difficulty →	Low	Medium	High
CRITICAL	—	—	—
HIGH	—	—	—
MEDIUM	—	● M-2	● ● M-1, M-3
LOW	● L-1	● ● ● L-2, L-4, L-5	● L-3
INFO	● I-1	—	—

Codebase Maturity

DOCUMENTATION Moderate Detailed architecture, integration, deployment, and timelock guidance; no dedicated threat model evidenced.	TESTING Moderate Covers handlers, debt swaps, collateral flows, Safe modules, and leverage; no coverage gates or invariants evidenced.	ACCESS CONTROLS Strong Timelock-gated critical changes; Safe modules restrict execution to the operator or Safe.
SUPPLY-CHAIN HYGIENE Moderate Lockfile-managed dependencies and established tooling; no automated vulnerability scanning evidenced.	ERROR HANDLING Strong SafeERC20, reentrancy and callback checks, fee caps, and emergency withdrawal.	DEPLOYMENT PROCESS Strong Resumable Ignition deployments with journals, address tracking, ABI export, and Base verification.

Findings Summary

ID	Title	Severity	Status	Fix commit
M-1	Pool-param delist bricks closeLp / collectLp swap legs	MEDIUM	FIXED	ba4a03f
M-2	MorphoDebtHandler.switchFrom always repays full borrowShares	MEDIUM	FIXED	790c6d5 , 0073c38
M-3	Gauge killed DoS on partial close of staked aerodrome positions	MEDIUM	FIXED	feff96b
L-1	closeLp reverts on sub-floor TWAP dust; collectLp skips	LOW	FIXED	959f266
L-2	Partial close: liquidity exits while basisForExit == 0	LOW	ACKNOWLEDGED	a2bbec9
L-3	V4 native switch residue TWAP key (WETH vs address(0))	LOW	FIXED	1a8907c
L-4	SafeDebtManager flash principal conversion rounds down	LOW	FIXED	06b28a7
L-5	V3/Aerodrome _chargeCollectFee mishandles empty returndata (USDT-style)	LOW	FIXED	db0d4d9
I-1	V4 allow-list does not enforce hooks == address(0) on-chain	INFO	ACKNOWLEDGED	6dbed01

Findings

M-1

Pool-param delist bricks closeLp / collectLp swap legs

Severity: MEDIUM · Difficulty: HIGH · Status: FIXED · Fix: ba4a03f (PR #19)

Target: SafeYieldManager, BaseYieldHandler, UniV4YieldHandler

Description

NatSpec on open-path token whitelist states exits must not brick on a later de-listing. That guarantee is **not** applied to **pool-param** allow-listing on exit swap legs.

`setPoolParamAllowed(..., false)` is callable immediately by `DEFAULT_ADMIN_ROLE` (no timelock). Exit paths still require the swap-leg `poolParam` to remain allow-listed via `_validateSwapLeg` → `_validatePoolParamAllowed`. Delisting a pool param used as a close/collect swap route makes the USDC-realizing exit revert while `withdrawLp` (no swap legs) still works.

Affected Code

```
///      ascending). Applies to openLp and the switchLp open leg only;
///      exits must never brick on a later de-listing.
function _requireWhitelistedPoolTokens(address handler, bytes calldata
lpPoolParam) internal view {
```

```
function setPoolParamAllowed(
    uint8 protocol,
    bytes calldata poolParam,
    bool allowed
) external onlyRole(DEFAULT_ADMIN_ROLE) {
```

```
function _validateSwapLeg(address token, SwapLeg calldata leg, uint16 slippageBps)
internal view {
    if (token == address(USDC)) return;
    ...
    _validatePoolParamAllowed(leg.poolParam);
```

(Same pattern in `UniV4YieldHandler._validateSwapLeg`; `closeLp` / fee-to-USDC `collectLp` call it before swapping.)

EXPLOIT SCENARIO

1. User opens LP with allow-listed pool param `P`.
2. Admin calls `setPoolParamAllowed(protocol, P, false)`.
3. `closeLp` / `collectLp` with swap leg `poolParam = P` → `PoolParamNotAllowed`.
4. `withdrawLp` for the same position still succeeds (in-kind).

IMPACT

Broken primary USDC exit for affected positions; NatSpec invariant violated. Workaround: in-kind withdraw or Safe owner calling NPM directly.

RECOMMENDATION

On exit paths, validate pair shape + TWAP floor only — do **not** require allow-list membership on swap legs.

Remediation (`ba4a03f`): Exit swap legs in `_validateSwapLeg` no longer call `_validatePoolParamAllowed`; opens re-apply the allow-list check in `_acquireSide`. De-listing a pool param stops new opens only and no longer bricks USDC exits on existing positions.

M-2

MorphoDebtHandler.switchFrom always repays full borrowShares

Severity: **MEDIUM** · Difficulty: **MEDIUM** · Status: **FIXED** · Fix: `790c6d5`, `0073c38` ([PR #19](#))

Target: `MorphoDebtHandler`

Description

`executeDebtSwap` flashes a caller-chosen `_amount` (any value ≥ 10000 , or `type(uint256).max` for full debt). Morpho `switchFrom` approves based on that `amount`, but always repays by **full** `borrowShares` from `extraData`, ignoring `amount` for repay sizing.

If `extraData` carries the position's full share count (typical) while the flash is only partial, Morpho needs more assets than the flash delivered → revert. Partial Morpho debt migration is effectively unusable.

Affected Code

```
function switchFrom(
    address fromAsset,
    uint256 amount,
    address onBehalfOf,
    CollateralAsset[] memory collateralAssets,
    bytes calldata extraData
) public override onlyAuthorizedCaller(onBehalfOf) {
    ...
    (MarketParams memory marketParams, uint256 borrowShares) =
    abi.decode(extraData, (MarketParams, uint256));
    ...
    IERC20(fromAsset).forceApprove(address(morpho), amount + (amount / 100));
    morpho.repay(marketParams, 0, borrowShares, onBehalfOf, "");
}
```

Contrast `repay()` in the same file, which can repay by assets when `borrowShares == 0`.

EXPLOIT SCENARIO

1. Operator calls `executeDebtSwap` with partial `_amount`.
2. `fromExtraData` encodes full live `borrowShares`.
3. Flash delivers `amount`; `switchFrom` tries to burn all `borrowShares`.
4. Repay reverts (insufficient assets) or over-consumes relative to flash sizing.

IMPACT

Broken partial Morpho switches; availability / integration defect. No unprivileged theft.

RECOMMENDATION

Derive repay shares from `amount` (`toSharesDown`); use full `borrowShares` only for `type(uint256).max` closes.

Remediation (`790c6d5` , `0073c38`): `switchFrom` and `repay` now size repayments via `_sizeRepayment` , repaying by shares only when the flash amount covers every live share after accrual, otherwise repaying exactly by assets. `getDebtAmount` accrues interest and quotes from the live position; `_closeShares` clamps stale calldata share counts to the on-chain balance (R-01).

M-3

Gauge killed DoS on partial close of staked aerodrome positions

Severity: **MEDIUM** · Difficulty: **HIGH** · Status: **FIXED** · Fix: `feff96b` (PR #19)Target: `BaseYieldHandler.sol`, `AerodromeYieldHandler.sol`

Description

When a user partially closes a staked Aerodrome CL position through `closeLp()`, the handler performs an **unstake** → **modify** → **restake** flow. The position is first withdrawn from the Aerodrome gauge (unstaked), the requested portion of liquidity is removed, and then the remaining position is deposited back into the gauge (restaked).

Aerodrome gauges can be killed by governance through the Voter contract. When a gauge is killed, `deposit()` functions revert with "GK" (Gauge Killed), while `withdraw()` continues to function. This creates a critical failure scenario during the restake step of a partial close, DoSing the partial closes for positions that are staked into killed gauges.

Affected Code

Unstaking from gauge:

```
address unstakedFrom = _unstakeIfStaked(p.onBehalfOf, p.tokenId);
_requireOwnedBy(p.onBehalfOf, p.tokenId);
```

```
function _unstakeIfStaked(address _onBehalfOf, uint256 tokenId) internal
override returns (address stakePool) {
    stakePool = _stakePoolOf(tokenId);
    if (stakePool == address(0)) return address(0);
    .
    .
    .
    address rewardToken = IStakePool(stakePool).rewardToken();
    uint256 rewardBefore = rewardToken == address(0) ? 0 :
IERC20(rewardToken).balanceOf(_onBehalfOf);
    _safeExec(_onBehalfOf, stakePool, abi.encodeCall(IStakePool.withdraw,
(tokenId)), 30);
    _settleStakedReward(_onBehalfOf, tokenId, rewardToken, rewardBefore);
}
```

Decreasing the liquidity:

```
if (liquidityToRemove > 0) {
    _safeExec(
        p.onBehalfOf,
        POSITION_MANAGER,
        abi.encodeCall(
            INonfungiblePositionManager.decreaseLiquidity,
            (
                INonfungiblePositionManager.DecreaseLiquidityParams({
                    tokenId: p.tokenId,
                    liquidity: liquidityToRemove,
```

```

        amount0Min: p.decreaseAmount0Min,
        amount1Min: p.decreaseAmount1Min,
        deadline: p.deadline
    })
    },
    7
);

```

Restaking into the gauge:

```

    if (unstakedFrom != address(0)) {
        if (p.exitBps == 10_000) {
            _clearStakePin(p.tokenId);
        } else {
=>            _restakeInto(p.onBehalfOf, p.tokenId, unstakedFrom);
        }
    }
}

```

`isAlive` check on the [gauge contract](#):

```

function deposit(uint256 tokenId) external override nonReentrant {
    require(nft.ownerOf(tokenId) == msg.sender, "NA");
=>    require(voter.isAlive(address(this)), "GK");
}

```

EXPLOIT SCENARIO

Setup: User stakes an Aerodrome CL position via RateHopper with `stake = true` in `openLp()`. After a while governance kills the gauge (e.g., pool deemed unsafe, zero votes, or malicious governance).

Trigger: User attempts partial close via `closeLp()` with `exitBps < 10000`.

Effect: Sequence executes: `_unstakeIfStaked()` → `IStakePool.withdraw()` → **SUCCESS** (NFT now on Safe); `decreaseLiquidity()` → **SUCCESS** (liquidity removed); `_restakeInto()` → `IStakePool.deposit()` → **REVERTS** with "GK".

IMPACT

Partial close functionality on pools with killed gauges become DoSed.

RECOMMENDATION

We recommend checking the gauge status prior to the restake operation. If the gauge is determined to be killed, the restake step should be skipped entirely.

Remediation (`feff96b`): New `_stakePoolAcceptsDeposits` hook (Aerodrome: `Voter.isAlive`). Partial closes on a killed gauge skip the restake, clear the stake pin, and emit `RestakeSkipped`; opens refuse to stake into a killed gauge.

L-1

closeLp reverts on sub-floor TWAP dust; collectLp skips

Severity: **LOW** · Difficulty: **LOW** · Status: **FIXED** · Fix: [959f266](#) ([PR #19](#))Target: `BaseYieldHandler.sol`, `UniV4YieldHandler.sol` — `_swapDeltaToUsdc()`

Description

After TWAP flooring, if `amountOutMin == 0`, behavior depends on `leaveZeroFloorInKind`: `collectLp` passes `true` (skip swap, leave dust in-kind); `closeLp` passes `false` (revert `InvalidSwapAmountOutMin`). Thin pools / dust deltas can turn a sub-floor residue into a total close failure.

Affected Code

```
_swapDeltaToUsdc(..., false); // closeLp
...
_swapDeltaToUsdc(..., true); // collectLp
...
if (amountOutMin == 0) {
    if (leaveZeroFloorInKind) return;
    revert InvalidSwapAmountOutMin();
}
```

(Same `false` on V4 `closeLp`; same gate in `_swapV4ViaSafe`.)

EXPLOIT SCENARIO

Setup: Position close produces a non-zero token delta whose TWAP quote floors to 0.

Trigger: `closeLp` hits `InvalidSwapAmountOutMin` and reverts fully.

Effect: Same residue on `collectLp` is skipped in-kind; USDC exit fails while harvest would succeed.

IMPACT

Availability / griefing on USDC exit. `withdrawLp` remains.

RECOMMENDATION

Pass `leaveZeroFloorInKind = true` from `closeLp` (match collect).

Remediation ([959f266](#)): `closeLp` now passes `leaveZeroFloorInKind = true` on V3/Aerodrome and V4 swap legs; sub-floor dust stays on the Safe in kind and is reported via the new `SwapSkippedBelowFloor` event. Open legs remain fail-closed.

L-2

Partial close: liquidity exits while basisForExit == 0

Severity: **LOW** · Difficulty: **MEDIUM** · Status: **ACKNOWLEDGED** · Fix: **a2bbec9** (PR #19)Target: `SafeYieldManager.sol`, `BaseYieldHandler.sol`, `UniV4YieldHandler.sol` — `closeLp()`

Description

Partial closes prorate basis and liquidity independently with `mulDiv(..., exitBps, 10_000)` (both round down). The only asymmetry guard rejects

`basisForExit > 0 && liquidityToRemove == 0`. The inverse —

`basisForExit == 0 && liquidityToRemove > 0` — is allowed, so liquidity leaves while recorded basis is unchanged and performance fee is charged on ~100% of realized USDC.

Affected Code

```
uint128 basisForExit = Math.mulDiv(uint256(residualBasis),
uint256(params.exitBps), 10_000).toUint128();
...
uint128 liquidityToRemove = p.exitBps == 10_000
    ? liquidity
    : Math.mulDiv(uint256(liquidity), uint256(p.exitBps), 10_000).toUint128();
if (p.exitBps != 10_000 && basisForExit > 0 && liquidityToRemove == 0) {
    revert InvalidExitBps();
}
...
if (realized > basisForExit) {
    uint256 profit = realized - uint256(basisForExit);
    feeUsd6 = ((profit * $.performanceFeeBps) / 10_000).toUint128();
```

EXPLOIT SCENARIO

Setup: Small residual basis (e.g. 1) and tiny `exitBps`, or large liquidity with tiny basis.

Trigger: Partial `closeLp` with `liquidityToRemove > 0`, `basisForExit == 0`.

Effect: Liquidity exits; basis storage unchanged; fee ≈ full realized value.

IMPACT

Performance-fee overcharge on dust / edge partial exits. No principal theft.

RECOMMENDATION

Also revert when `basisForExit == 0 && liquidityToRemove > 0` (unless full close).

Remediation (**a2bbec9**): Accepted without code change. The omitted basis slice is below one raw USDC unit per partial close and stays in the remaining basis; a zero basis is also a legitimate

post-switch state whose partial closes the suggested revert would block. Documented in [docs/SECURITY_MODEL.md](#) (Shred September 2026 L-2).

L-3

V4 native switch residue TWAP key (WETH vs address(0))

Severity: **LOW** · Difficulty: **HIGH** · Status: **FIXED** · Fix: [1a8907c](#) ([PR #19](#))Target: `UniV4YieldHandler.sol`, `SafeYieldManager.sol` — `withdrawLp()`, `_valueInUsd6()`

Description

Native V4 pools use `currency0 = address(0)`. Allow-list / swap TWAP paths can be keyed at `twapConfigOf[address(0)]`. On `withdrawLp`, native ETH is wrapped and returned as **WETH**. `switchLp` then values residue via `_valueInUsd6(token)` with no `WETH → address(0)` alias — incomplete TWAP seeding causes `TwapNotConfigured(WETH)`.

Affected Code

```
token0 = key.currency0;
token1 = key.currency1;
if (key.currency0 == NATIVE) {
    if (amount0 > 0) {
        _safeExecValue(p.onBehalfOf, address(WETH), amount0,
abi.encodeCall(IWETH9.deposit, ()), 73);
    }
    token0 = address(WETH);
}
```

```
function _valueInUsd6(address token, uint256 amount) internal view returns
(uint256) {
    if (amount == 0) return 0;
    if (token == address(USDC)) return amount;
    TwapConfig memory cfg = _yieldStorage().twapConfigOf[token];
    if (cfg.pool == address(0)) revert TwapOracle.TwapNotConfigured(token);
}
```

EXPLOIT SCENARIO

Setup: Deploy/allow-list native V4 pool with only `twapConfigOf[address(0)]` set. Open native V4 LP.

Trigger: Later `switchLp` leaves WETH residue; `_settleSwitchResidue` → `_valueInUsd6(WETH, ...)`.

Effect: Reverts `TwapNotConfigured(WETH)`.

IMPACT

DoS on `switchLp` from native V4 positions under incomplete TWAP seeding. Production deploy seeds both keys — latent misconfig, not current mainnet outage.

RECOMMENDATION

Alias `WETH → address(0)` in `_valueInUsd6`, or require both keys when allow-listing native V4 pools.

Remediation (`1a8907c`): Allow-listing a native V4 pool now requires both `twapConfigOf[address(0)]` and `twapConfigOf[WETH]` on the constructor, `setPoolParamAllowed`, and the open re-enable path — incomplete seeding fails at configuration time instead of on the first switch.

L-4

SafeDebtManager flash principal conversion rounds down

Severity: **LOW** · Difficulty: **MEDIUM** · Status: **FIXED** · Fix: **06b28a7** (PR #19)Target: **SafeDebtManager.sol** — **uniswapV3FlashCallback()**

Description

When `paraswapParams.srcAmount == 0` and `fromAsset` has more decimals than `toAsset` (e.g. WETH→USDC), principal is converted with **floor** division while the flash fee is converted with **ceil** division. That understates swap sizing relative to exact flash repay needs and can brick the debt switch.

Affected Code

```
if (decimalDifference > 0) {
    // Round up: (a + b - 1) / b
    uint divisor = 10 ** uint256(decimalDifference);
    flashloanFee = (flashloanFeeOriginal + divisor - 1) / divisor;
} else if ...
uint256 amountInToAssetDecimals = decoded.amount;
if (decimalDifference > 0) {
    amountInToAssetDecimals = decoded.amount / (10 ** uint256(decimalDifference));
// floor
} else if ...
uint256 amountInMax = decoded.paraswapParams.srcAmount == 0
    ? amountInToAssetDecimals
    : decoded.paraswapParams.srcAmount;
...
uint256 amountToRepay = decoded.amount + flashloanFeeOriginal;
swapByParaswap(..., amountToSwap, amountToRepay, ...);
```

EXPLOIT SCENARIO

Setup: Debt switch with `srcAmount == 0`, `fromAsset` decimals > `toAsset`.

Trigger: Floored principal + ceiled fee → swap cannot cover exact flash repay.

Effect: Revert `InsufficientTokenBalanceAfterSwap`.

IMPACT

DoS on affected cross-decimal debt switches when relying on auto `srcAmount`.

RECOMMENDATION

Ceil the principal conversion the same way as the fee (or size the swap from exact repay needs).

Remediation (**06b28a7**): Principal and flash-fee decimal conversion now share one branch; both round up with `Math.ceilDiv` when the source asset has more decimals than the

destination, so auto-sized swaps (`srcAmount == 0`) no longer under-size relative to the exact flash repayment.

L-5

V3/Aerodrome _chargeCollectFee mishandles empty returndata (USDT-style)**Severity:** LOW · **Difficulty:** MEDIUM · **Status:** FIXED · **Fix:** db0d4d9 (PR #19)**Target:** BaseYieldHandler.sol — _chargeCollectFee()

Description

_chargeCollectFee skims feeCollectBps with a high-level try IERC20.transfer(...) returns (bool ok). Tokens that return **empty** returndata on success (USDT-style) make ABI decoding of that bool fail. Control enters catch even though the treasury transfer already ran; the subsequent safeTransfer of the full amount reverts and rolls back the tx.

Affected Code

```

if (fee > 0) {
    try IERC20(token).transfer($.treasury, fee) returns (bool ok) {
        if (ok) {
            toSafe = amount - fee;
        } else {
            emit CollectFeeTransferFailed(...);
            fee = 0;
        }
    } catch {
        emit CollectFeeTransferFailed(...);
        fee = 0;
    }
}
if (toSafe > 0) IERC20(token).safeTransfer(_onBehalfOf, toSafe);

```

UniV4 and Aerodrome reward skims already treat empty returndata as success via TokenReturnLib.returnedTrue / _trySafeTokenTransfer; this LP skim does not. Only matters when feeCollectBps > 0 and the harvested token is no-return.

EXPLOIT SCENARIO

Setup: Position accrues fees in a no-return token; feeCollectBps > 0.

Trigger: Harvest calls _chargeCollectFee; treasury transfer succeeds with empty returndata.

Effect: catch runs; toSafe stays amount; safeTransfer reverts → outer collectLp / fee-harvesting closeLp / withdrawLp reverts.

IMPACT

Availability only for that fee token. No fund theft. Not a silent protocol fee leak (tx does not complete with fee waived). Scope is allow-listed pools whose fee side is a no-return ERC20.

RECOMMENDATION

Use `SafeERC20.safeTransfer` (or low-level call + `TokenReturnLib.returnedTrue`) for the treasury skim, and only reduce `toSafe` when that transfer succeeds; keep waive-on-failure semantics for real failures.

Remediation (`db0d4d9`): Treasury skim now uses `IERC20.trySafeTransfer`, accepting empty returndata from no-return tokens while still waiving the fee on false returns, short returndata, or reverts.

I-1

V4 allow-list does not enforce hooks == address(0) on-chain

Severity: **INFORMATIONAL** · Difficulty: N/A · Status: **ACKNOWLEDGED** · Fix: **6dbed01** ([PR #19](#))

Target: **SafeYieldManager.sol**, **UniV4YieldHandler.sol** — **setPoolParamAllowed()**

Description

V4 pool params are gated solely by the admin allow-list. There is no on-chain check that

PoolKey.hooks == address(0). A hooked pool's **beforeModifyLiquidity** /

afterModifyLiquidity runs on every module liquidity path, including oracle-free **withdrawLp**.

Affected Code

Production is hookless today; risk is admin mis-allow-list (or future hooked-pool policy without extra guards). Hostile/reverting hooks can brick module exits for that pool; Safe owners retain direct NPM exit.

Impact

No live exposure under current hookless allow-list. Admin misconfiguration risk for future pool additions.

RECOMMENDATION

```
PoolKey memory key = abi.decode(poolParam, (PoolKey));
if (key.hooks != address(0)) revert HookedPoolsNotSupported();
```

in the V4 allow-list path until a vetted hook registry + permission-bit policy exists.

Remediation (**6dbed01)**: Acknowledged with on-chain hardening rather than a blanket ban.

Hooked V4 pools stay supported behind the allow-list, but **setPoolParamAllowed** and the

constructor now refuse hooked keys (**HookedPoolParamNeedsTimelock**); only the timelocked

allowHookedPoolParam admits one. Exit swap legs may route through a hooked pool only if it is allow-listed; handlers expose the distinction via **IYieldHandler.poolParamHasHooks**.

Appendix A — Deployment Checklist Compliance

We compare every protocol's deployment against our published [Protocol Deployment Checklist](#)

Verdict: Mostly followed — strong on technical readiness, gaps in operational governance.

The project demonstrates solid **pre-deployment review, deployment scripting, and testing** practices. However, several **monitoring, incident response, and admin key management** items are missing or only partially documented. Overall compliance is **~75–80%**.

What is followed

Pre-deployment (strong)

Check	Status	Evidence
Audits by independent party	 Yes	[Shred Security](https://shredsecurity.io)
Threat model & deployment risks	 Yes	docs/SECURITY_MODEL.md — custody model, fees, emergency powers, trust assumptions, recovery procedures
MEV / frontrunning evaluated	 Yes	TWAP floor enforcement documented in SECURITY_MODEL.md H-01; swap slippage protection
Critical steps separated	 Yes	Timelock 2-day delay for CRITICAL_ROLE ops; constructor-time seeding vs post-deploy changes
Recovery plan exists	 Yes	SECURITY_MODEL.md I-03 recovery section; rescueToken, rescueERC721, setPauser rotation
All privileged roles documented	 Yes	README, SECURITY_MODEL.md: DEFAULT_ADMIN_ROLE, CRITICAL_ROLE (timelock), pauser, safeOperator
Fork-based mainnet tests	 Yes	4 fork test suites: safeYieldManagerUniV3Fork.ts, safeYieldManagerAerodromeFork.ts, safeYieldManagerUniV4Fork.ts, safeYieldManagerSwitchFork.ts
Emergency pause tested	 Yes	Pausable integration; whenNotPaused on open paths; exits deliberately pause-exempt
Reentrancy protection	 Yes	nonReentrant modifier on state-changing functions (6 contracts import ReentrancyGuard)

Architecture (N/A for proxy items)

Check	Status	Evidence
Upgradeability pattern documented	N/A	Immutable contracts, no proxies
Proxy admin not EOA	N/A	No proxies used
Initialization one-time only	N/A	No initializers — constructor-based
Re-initialization impossible	N/A	No reinitializer patterns found

Deployment tooling (strong)

Check	Status	Evidence
Deployment scripts	 Yes	Hardhat Ignition modules: <code>1_DeployCore.ts</code> , <code>2_DeployYieldManager.ts</code> , <code>TimelockControllerModule.ts</code>
Script outputs addresses	 Yes	<code>deployed_addresses.json</code> written to <code>ignition/deployments/chain-<chainId>/</code>
Constructor args recorded	 Yes	<code>journal.jsonl</code> captures all constructor arguments for verification
Network/chainId correct	 Yes	<code>hardhat.config.ts</code> correctly configures Base mainnet (8453)
Sequential transaction chaining	 Yes	Ignition <code>after</code> dependencies prevent nonce race conditions

Verification (adequate)

Check	Status	Evidence
Explorer verification script	 Yes	<code>scripts/verifyAll.ts</code> — reads from Ignition state, handles Etherscan V2 API bug
Deployed addresses documented	 Yes	<code>deployments/base.json</code> with 4 yield contracts
Sourcify enabled	 Yes	<code>hardhat.config.ts</code> line 43: <code>sourcify: { enabled: true }</code>

Testing (strong)

Check	Status	Evidence
CI pipeline	✓ Yes	<code>.github/workflows/ci.yml</code> — lint, compile, coverage, branch-coverage gate
Fork tests with pinned block	✓ Yes	<code>BASE_FORK_BLOCK_NUMBER=49470000</code> for deterministic runs
Unit/integration tests	✓ Yes	Comprehensive suites in <code>test/debt/</code> , <code>test/yield/</code> , <code>test/legacy/</code>

What is NOT followed (or only partial)

BLOCKER gaps

Item	Status	Notes
Verification completed on all chains	⚠ Partial	Only Base deployed; <code>deployments/base.json</code> exists but no per-chain verification evidence log
Bytecode matches audited build	⚠ Not evidenced	No automated reproducible-build verification documented
External token addresses verified	⚠ Implicit	<code>contractAddresses.ts</code> has addresses; no explicit verification table against canonical sources

WARNING gaps (material)

Item	Status	Notes
Admin roles held by multisig	⚠ Documented but not verified	SECURITY_MODEL.md: "The pauser must be a multisig, not an EOA" — but no on-chain verification step
Multisig signers validated	⚠ Not in repo	Operator-side responsibility
Alerts for admin/role changes	✗ Not set up	SECURITY_MODEL.md: "alert on it" — no actual monitoring/alerting infrastructure
Economic anomaly alerts	✗ Not set up	No monitoring mentioned
Secure backups documented	✗ Not in repo	Operator-side responsibility
State verified across multiple RPCs	✗ Not automated	No multi-RPC sanity check script
Deployment simulated in Tenderly/Foundry fork	⚠ Partial	Fork tests exist but no pre-deploy simulation documented

INFO gaps

Item	Status	Notes
Incident response runbook	⚠ Partial	Recovery procedures in SECURITY_MODEL.md I-03; not a formal runbook with playbooks
Emergency drills scheduled	✗ Not documented	No evidence of scheduled drills
User communication plan	✗ Not documented	No incident communication plan
Dashboards live and ingesting	✗ Not set up	No analytics/dashboards

Section-by-section score

Section	Followed?
1.1 Architecture & upgradeability	N/A (no proxies — immutable design)
1.2 Threat model & deployment risks	Mostly — MEV sim not evidenced
1.3 Audits & testing	Yes — 3+ audits, fork tests, CI
1.4 External asset validation	Partial — addresses exist, no explicit verification table
2.1 Deployment script sanity	Yes — Ignition modules with proper chaining
2.2 Proxy deployment & init	N/A (no proxies)
2.3 Explorer verification	Mostly — script exists, no per-chain evidence trail
2.4 Admin & key management	Partial — multisig documented, not verified; backups not documented
3.1 On-chain sanity validation	N/A (no proxies) / Partial for general state checks
3.2 Functional smoke tests	Manual — fork tests exist, no post-deploy canary script
3.3 Monitoring & alerting	No — not set up
3.4 Emergency procedures	Partial — recovery in docs, no formal runbook

Deployed Contracts (Base mainnet)

Contract	Address
SafeYieldManager	0x93281e160181B8EBdEB8a8d9493BcD34dE5fA1b8
UniV3YieldHandler	0x057790539471b103012df78e5A923E0b41d86168
AerodromeYieldHandler	0x942c5A46570F31236a6560aDE8Fd740A5163e303
UniV4YieldHandler	0x748b064A61f30E41b0904f91e5874f59714777d3

Bottom line

For **smart contract technical readiness: strong** — audited (multiple reports), tested (fork + unit + CI), deterministic deployment, proper access control, timelock on critical ops, reentrancy guards, documented threat model.

For **operational deployment governance: gaps remain:**

1. No monitoring/alerting infrastructure
2. No formal incident response runbook (recovery documented, but scattered)
3. Multisig requirement stated but not on-chain verified
4. No reproducible-build / bytecode-match verification step
5. External token addresses implicit, not explicitly verified against canonical sources

Practical rating: ~75–80% compliant — very strong on smart-contract security and deployment scripting; missing operational observability and formal incident procedures.

Appendix B — Stateful Invariant Fuzzing Report

Target: PR #16 (`feat/support-aerodrome` / SafeYieldManager yield stack)

Repository: `RateHopper/ratehopper-contracts`

Commit under test: `1493cbc` (branch `audit/pr-16`)

Base for delta review: `main` @ `f17d601`

Report date: 27 August 2026

Engagement type: Automated invariant fuzzing (Fizz / Medusa / Echidna / Foundry)

Status: Complete — no property violations observed

1. Executive Summary

This report documents a stateful fuzzing campaign against the RateHopper SafeYieldManager yield stack introduced in PR #16, including Uniswap V3, Aerodrome Slipstream, and Uniswap V4 handlers, plus a focused SafeDebtManager I-02 residue-refund path.

We generated and executed **24 specification-level properties** (23 implemented, 1 intentionally deferred). Campaigns ran under **Medusa** (~1.14M transactions over ~60 minutes), **Echidna** (~50k transactions with Slither analysis enabled), and **Foundry** smoke plus event-based fuzz tests for fee/residue identities that cannot be checked inside Medusa/Echidna.

Key results

Metric	Result
Property / assertion failures (Medusa 1h)	0 (27/27 tests passed)
Property / assertion failures (Echidna)	0 (0/19 failed)
Foundry suite (<code>FoundryTester</code>)	6/6 passed (incl. 2×256-run event fuzzes)
Confirmed protocol bugs from this campaign	None
Deferred / out-of-harness checks	SP-06 (router minOut instrumentation); native-ETH V4 symmetry; full flash-loan debt swap

Interpretation. Absence of violations under these campaigns increases confidence that the checked accounting, pause/exit, and non-custodial invariants hold on the mock-backed harness. It does **not** by itself constitute a full security audit of live mainnet integrations, oracle manipulation under adversarial pool state, or Paraswap/UniversalRouter path correctness.

2. Scope

2.1 In scope (fuzzing focus)

Component	Role
<code>SafeYieldManager.sol</code>	Safe module entry: open / close / withdraw / switch / collect, fees, TWAP, pause
<code>UniV3YieldHandler.sol</code>	Uniswap V3 LP handler
<code>AerodromeYieldHandler.sol</code>	Aerodrome Slipstream + stake surface
<code>UniV4YieldHandler.sol</code>	Uniswap V4 standalone handler
<code>BaseYieldHandler.sol</code> / <code>V3StyleYieldHandler.sol</code>	Shared V3-style flow (via delegatecall)
<code>TwapOracle.sol</code> / <code>YieldStorage.sol</code>	Pricing + ERC-7201 accounting
<code>SafeDebtManager.exit</code>	I-02 post-repay residue refund (declining-repay mock)

Primary reference: `Audit docs/scope.md` , `docs/SECURITY_MODEL.md` .

2.2 Explicitly limited / out of campaign scope

- Full greenfield re-audit of pre-existing debt handlers (Aave/Compound/Morpho/Moonwell/Fluid) beyond the I-02 residue pattern
- Live Base mainnet fork against production Uniswap / Aerodrome / V4 deployments
- Paraswap routing correctness and MEV on swap legs
- Native-ETH Uniswap V4 open/withdraw symmetry (ERC20 V4 pool registered; native key allow-listed but not stress-fuzzed as a primary flow)
- SP-06: proving `amountOutMin >= twapMinimumOut` on the internal router call (mock does not expose the received minOut)

2.3 Trust model assumptions (accepted)

Per `docs/SECURITY_MODEL.md` , the harness treats as trusted: `safeOperator` , pauser, `DEFAULT_ADMIN_ROLE` , timelock+ `CRITICAL_ROLE` , and correctly configured TWAP seeds / pool allow-lists. Properties that assume honest oracle configuration are tagged accordingly.

3. Methodology

3.1 Approach

The campaign follows the **Fizz** stateful fuzzing workflow:

1. Protocol understanding and entry-point selection (`fizz_data/protocol-understanding.md` , `entry-point-selection.json`)
2. Harness generation under `test/fizz/` (actors, mocks, clamped handlers, ghosts/snapshots)
3. Invariant discovery → `PROPERTIES.md` / `property-plan.md` with **Guarantee** tags:
 - **SHOULD-HOLD** — documented or exact mathematical identity; violation ⇒ confirmed bug (after ruling out harness error)
 - **EXPLORATORY** — inferred; violation ⇒ human-review lead
4. Dual-fuzzer execution (Medusa primary; Echidna complementary) plus Foundry for event-parse properties
5. Triage, report, and `fizz_sync` snapshot (`fizz_data/last-run.json`)

3.2 Harness architecture

Artifact	Path
Fuzzer entry	<code>test/fizz/FuzzTester.sol</code>
Foundry entry	<code>test/fizz/FoundryTester.sol</code>
Setup / ghosts	<code>test/fizz/Base.sol</code>
Global properties	<code>test/fizz/Properties.sol</code>
Yield handlers	<code>test/fizz/handlers/SafeYieldManagerHandler.sol</code>
Debt I-02 handler	<code>test/fizz/handlers/SafeDebtManagerHandler.sol</code>
Spec English	<code>PROPERTIES.md</code>

Fixture: Mock ERC20s (`MockWETH` + USDC), mock UniV3 / Aerodrome / UniV4 stacks (`Ratehopper*Mocks`), `MockSafeHarness` , `MockRegistry` , real `SafeYieldManager` + production handler bytecode. Protocols registered: UniV3 (0), Aerodrome (1), UniV4 (2). TWAP seeds for WETH and native (`address(0)`) point at the mock UniV3 pool (tick 0 ⇒ 1:1 pricing for residue ghosts).

Debt I-02: `MockResidueDebtHandler` leaves 1 wei undeclared on `repay` under delegatecall; `exit_withResidue_clamped` asserts the manager ends with zero USDC and the Safe retains the leave-behind.

3.3 Tooling

Tool	Version / notes
Foundry (<code>forge</code>)	1.5.1
Medusa	1.5.1
Echidna	2.2.6
Slither	0.11.6 (enabled for Echidna/Medusa analysis)
cryptic-compile	0.4.2 (+ Foundry build-info shim for <code>*.output.json</code>)

3.4 Campaign configurations

Campaign	Config highlights	Outcome
Medusa 1h	<code>timeout 3600</code> , <code>testLimit 0</code> , 10 workers, assertion + <code>property_*</code>	27/27 PASS
Medusa prior	<code>testLimit 500000</code> (~25m)	27/27 PASS
Echidna	<code>testLimit 50000</code> , assertion mode, Slither on	0/19 FAIL
Foundry	<code>FoundryTester</code> + 256-run fuzzes	6/6 PASS

4. Findings

4.1 Summary of findings

#	Title	Severity	Status
—	<i>No property violations attributed to protocol logic</i>	—	—

No SHOULD-HOLD property was violated under Medusa, Echidna, or Foundry event tests. No EXPLORATORY property produced a failing trace requiring triage.

4.2 Coverage gaps (not findings)

These are **limitations of the campaign**, not confirmed defects:

1. **SP-06 (TWAP floor on router minOut)** — not instrumented; remains `[-]` in `PROPERTIES.md` .
2. **Native-ETH V4 path** — allow-listed; not a primary fuzz sequence.

3. **Flash-loan debt swap residue** — I-02 covered via `exit()`; Uniswap V3 flash callback path not fully harnessed.
4. **Mock TWAP / routers** — adversarial AMM manipulation and real router semantics are out of band.

5. Property Catalog

Guarantee legend: **SH** = SHOULD-HOLD · **EX** = EXPLORATORY

Status legend: **Pass** = implemented and no violation in campaigns · **N/A** = deferred / not runnable in fuzzer

5.1 Global properties (`property_*` , checked by Medusa after sequences)

ID	Property	Cat.	G	Priority	Solidity	Campaign result
GL-01	Cleared positions have zero basis and carry	HIGH_LEVEL	SH	HIGH	<code>property_clearedPositionsHaveNoAccounting</code>	Pass
GL-02	Every live ghost tokenId has a non-zero pin	HIGH_LEVEL	SH	HIGH	<code>property_liveIdsArePinned</code>	Pass
GL-03	Manager USDC/WETH balances are zero after ops	HIGH_LEVEL	SH	HIGH	<code>property_managerHoldsNoUserTokens</code>	Pass
GL-04	Live positions remain pinned until full exit	VALID_STATE	SH	HIGH	<code>property_pinnedImpliesKnownPosition</code>	Pass
GL-05	Pause does not wipe live pins/basis	VALID_STATE	EX	MED	<code>property_pauseDoesNotWipeBasis</code>	Pass
GL-06	Σ live residual basis (V3+Aero+V4) $\leq$ Σ opened basis	HIGH_LEVEL	SH	HIGH	<code>property_liveBasisBoundedByOpens</code>	Pass
GL-07	Switch: <code>newCarry - newBasis = oldCarry - oldBasis + U</code>	HIGH_LEVEL	SH	HIGH	<code>property_switchResidueConservation</code>	Pass
GL-08	Partial close: taken + remaining = original (basis & carry)	HIGH_LEVEL	SH	HIGH	<code>property_partialCloseProration</code>	Pass
I-02	Debt manager holds zero USDC after residue exit	HIGH_LEVEL	SH	HIGH	<code>property_debtManagerHoldsNoUsdc</code>	Pass

5.2 Specific / handler-inline properties

ID	Property	Cat.	G	Priority	Where checked	Campaign result
SP-01	<code>switchLp</code> clears source pin/basis/carry	STATE_TRANSITION	SH	HIGH	<code>switchLp_clamped</code> asserts	Pass
SP-02	<code>switchLp</code> pins destination	STATE_TRANSITION	SH	HIGH	<code>switchLp_clamped</code>	Pass
SP-03	<code>newBasis ≤ oldBasis</code> after switch	VARIABLE_TRANSITION	SH	HIGH	<code>switchLp_clamped</code>	Pass
SP-04	Residue USD equals independent <code>twapQuote</code>	HIGH_LEVEL	SH	HIGH	Foundry <code>testFuzz_sp04_*</code> (events)	Pass (256 runs)
SP-05	Perf fee = $(\text{value} + \text{carry} - \text{basis}) \times \text{feeBps} / 10000$ when profitable	STATE_TRANSITION	SH	HIGH	Foundry <code>testFuzz_sp05_*</code> (events)	Pass (256 runs)
SP-06	Router <code>amountOutMin ≥ twapMinimumOut</code>	VARIABLE_TRANSITION	SH	HIGH	—	N/A (deferred)
SP-07	<code>closeLp</code> succeeds while globally paused	VALID_STATE	SH	MED	<code>closeLp_whilePaused</code>	Pass
SP-08	UniV4 open/close(/switch) lifecycle	STATE_TRANSITION	EX	MED	V4 handlers + smoke	Pass
SP-09	<code>newCarry ≥ oldCarry</code> across switch	VARIABLE_TRANSITION	SH	HIGH	inline assert	Pass
SP-10	<code>collectLp</code> does not mutate basis/carry	VARIABLE_TRANSITION	SH	HIGH	inline assert	Pass
SP-11	No fee when unprofitable close	STATE_TRANSITION	SH	HIGH	<code>property_noFeeWhenUnprofitable</code>	Pass
SP-12	Safe balances non-decreasing on switch	HIGH_LEVEL	SH	MED	<code>property_safeBalsNonDecreasingOnSwitch</code>	Pass
SP-13	No fee on switch / withdraw	STATE_TRANSITION	SH	HIGH	<code>property_noFeeOnNonCloseExit</code>	Pass

5.3 Adversarial properties

ID	Property	Cat.	G	Priority	Where checked	Campaign result
ADV-01	Partial-close basis slices + remaining = original	HIGH_LEVEL	SH	HIGH	<code>property_partialCloseBasisConservation</code>	Pass
ADV-03	<code>withdrawLp</code> works when TWAP/oracle broken	VALID_STATE	SH	MED	<code>withdrawLp_whenBlocked</code>	Pass
ADV-08	Zero-residue dust switch preserves basis	HIGH_LEVEL	SH	HIGH	<code>switchLp_dust</code> / <code>property_dustSwitchPreservesBasis</code>	Pass

5.4 Property distribution

Dimension	Count
Total specs in <code>PROPERTIES.md</code>	24 (+ I-02 Medusa property)
Implemented <code>[x]</code>	23
Deferred <code>[-]</code>	1 (SP-06)
SHOULD-HOLD	21 (incl. strengthened GL-06)
EXPLORATORY	2 (GL-05, SP-08)
Checked in Medusa globals	9 <code>property_*</code>
Checked Foundry-only (events)	SP-04, SP-05

6. Campaign Evidence

6.1 Medusa (1-hour production run)

Field	Value
Log	<code>fizz_data/medusa-1h-campaign.log</code>
Wall time	~59m59s fuzz elapsed
Transactions	~1,138,785
Branches hit	7,607
Corpus size	71
Property failures	0
Test summary	27 passed, 0 failed
Coverage HTML	<code>fizz_data/corpus_medusa/coverage/coverage_report.html</code>

Passed Medusa surfaces included: all clamped yield handlers, V4 open/close/switch, `exit_withResidue_clamped`, admin pause paths, and all `property_*` globals listed in §5.1.

6.2 Echidna (complementary, Slither enabled)

Field	Value
Log	<code>fizz_data/echidna-slither-campaign.log</code>
Unique instructions	72,684
Codehashes	43
Corpus	12
Calls	50,280
Failures	0/19

6.3 Foundry

Test	Result
<code>test_smoke_openClose</code>	PASS
<code>test_smoke_switchResidue</code>	PASS
<code>test_smoke_openCloseV4</code>	PASS
<code>test_smoke_debtI02_residue</code>	PASS
<code>testFuzz_sp04_switchResidueMatchesTwap</code>	PASS (256 runs)
<code>testFuzz_sp05_perfFeeFormula</code>	PASS (256 runs)

SP-04/SP-05 parse `SwitchResidueSettled` / `PositionClosed` via `vm.recordLogs()` and recompute expected values with `twapQuote` / fee formula — the same identities that Medusa cannot verify from events alone.

7. Recommendations

7.1 For protocol owners

1. Treat this report as **complementary evidence** to manual review of UniV4 handler edge cases (Permit2, native ETH, UniversalRouter) and debt flash-callback residue on real handlers.
2. Keep performance-fee and residue identities covered in CI via the Foundry event tests (`SP-04` / `SP-05`).
3. If TWAP-floor enforcement on opens is a release gate, instrument the mock (or a wrapper) so **SP-06** can be asserted.

7.2 For continued fuzzing

1. Optional longer Medusa (e.g. 4h) for additional corpus diversity — diminishing returns after 1.1M clean calls, but useful before mainnet deploy.
2. Add native-ETH V4 open/withdraw handlers if that product path ships.
3. Extend I-02 to the Uniswap V3 flash callback path with declining real handlers (Aave 1-wei pattern).
4. Prefer a via-IR fuzz profile when available for less deflated branch metrics.

7.3 Reproduction commands

```
export PATH="$HOME/.local/bin:$HOME/.foundry/bin:$PATH"
cd /path/to/ratehopper-contracts

forge test --match-contract FoundryTester -vv

node ~/.claude/skills/fizz/scripts/run_medusa.js . --meta-dir fizz_data --timeout 3600
# or: medusa fuzz

node ~/.claude/skills/fizz/scripts/run_echidna.js . --meta-dir fizz_data --timeout 600
# or: echidna . --contract FuzzTester --config echidna.yaml
```

8. Appendix A — Entry points exercised

Primary (yield): `openLp_clamped`, `closeLp_clamped`, `withdrawLp_clamped`,
`switchLp_clamped`, `collectLp_clamped`, `closeAero_clamped`, `openLpV4_clamped`,
`closeV4_clamped`, `switchToV4_clamped`

Liveness / adversarial: `closeLp_whilePaused`, `withdrawLp_whenBlocked`, `switchLp_dust`

Debt: `exit_withResidue_clamped`

Secondary admin: `admin_pause`, `admin_unpause`, `admin_setProtocolEnabledForClose`,
`setCurrentActor`, `openLp_unclamped`

9. Appendix B — Document map

Document	Purpose
<code>PROPERTIES.md</code>	Authoritative English property list + Guarantee tags
<code>fizz_data/property-plan.md</code>	Implementation wiring (may lag checkbox status; trust <code>PROPERTIES.md</code>)
<code>fizz_data/protocol-understanding.md</code>	Protocol notes used for harness design
<code>Audit docs/scope.md</code>	Audit scope for PR #16
<code>docs/SECURITY_MODEL.md</code>	Accepted risks / invariant language
<code>fizz_data/last-run.json</code>	Fizz sync snapshot for drift detection

10. Disclaimer

This report describes the results of automated property-based testing against a **mock-backed** harness. It does not replace a full manual security audit, formal verification, or economic analysis under live market conditions. Properties tagged EXPLORATORY are hypotheses; SHOULD-HOLD properties are only as strong as the cited documentation and the fidelity of mocks to production dependencies.

Final Notes

Remediation for all nine findings was delivered in [PR #19](#) (`fix/audit-shred-20260904` , head `bbef58a`). **7 Fixed, 2 Acknowledged** (L-2 documented accept; I-1 timelock hardening). See the Findings Summary table and per-finding remediation blocks for commit references.

This report was prepared by Shred Security. The sole canonical source is shredsecurity.io. Copies accessed through other channels may have been modified.